

Per-Operator Attribution of the Fake-Quantization-to-Integer Gap in INT8 Inference

[AUTHOR NAME]

Abstract—INT8 deployment decisions are made on a simulator. The network runs in floating point with quantize–dequantize (“fake-quantization”) nodes inserted, and its accuracy stands in for the accuracy of the integer target. We measure how far that stand-in holds — for accuracy and for tensors separately — against an integer engine that is bit-exact with the TensorFlow Lite reference kernels. On six networks, two quantization schemes and complete test splits, it holds for accuracy: every fake-versus-integer difference is within 0.20 pp and all twelve 95% confidence intervals contain zero. It fails for tensors: 28.5–72.7% of the output codes differ, on 95–100% of the images. Teacher forcing attributes the divergence per operator. Convolution, linear and pooling each disagree on under 1% of their own codes, while integer LayerNorm disagrees on 24.8%. Making the simulator integer-faithful at LayerNorm removes that source entirely (24.77% $\rightarrow$ 0.00%) but moves the propagated output mismatch only from 72.7% to 65.7%: a simulator cannot be repaired into a source of bit-accurate vectors one operator at a time. We also price a requantization choice the accuracy proxy cannot see — replacing rounding with truncation costs 0.90–3.37 pp in all 18 (model, seed, mode) combinations, and the sign of that cost transfers across seeds while its magnitude does not.

Index Terms—Quantization, integer arithmetic, embedded inference, neural processing units, reproducibility.

I. INTRODUCTION

TWO programs decide an INT8 deployment. The one that answers questions is a floating-point graph with quantize–dequantize (“fake-quantization”) nodes, whose accuracy is reported as the quantized accuracy [1], [2]. The one that ships is an integer program: int8 weights, an int32 accumulator, and a fixed-point multiply-and-shift requantization [3]. Practice treats the two as interchangeable, and for accuracy they largely are [4].

Accuracy is not all that is asked of the simulator. Silicon bring-up and compiler validation need *golden vectors*: the exact integer tensors the device must produce. Triaging a post-training quantization regression needs to know *which* operator started the divergence. Both are tensor questions, and to our knowledge neither has been measured. We ask three. **RQ1**: does agreement on accuracy imply agreement on tensors? **RQ2**: which operators create the divergence, and does removing the largest one close it? **RQ3**: what does the accuracy proxy fail to price?

Answering them needs a decomposition that current layer-wise error analyses do not provide: separating what an operator’s own integer arithmetic contributes (*local mismatch*,

isolated by teacher forcing) from what survives end-to-end execution (*propagated mismatch*). That decomposition, reported with paired standard errors on full test splits, is this letter’s methodological contribution; Findings 1–5 are its empirical ones.

II. BACKGROUND AND RELATED WORK

A fake-quantization graph rounds and clips a tensor to the quantization grid and then returns to float, so every operator still runs in float32. The integer program instead keeps int8 codes end to end and rescales with a Q31 multiplier and a shift [1], [3]. The two therefore agree on *intent* and are free to disagree on every least-significant bit.

MQBench [5] measures the gap between academic quantization and deployable backends and reports it as accuracy. The quantization white papers [4], [2] treat fake quantization as the simulation of the integer pipeline. Layer-wise error analyses in deployment toolchains quantify how far a quantized layer is from its *float* counterpart. Our measurement is different in kind: how far the *simulator* is from the *integer execution* the device performs, in codes rather than in signal-to-noise ratio. Integer-only transformer inference has established integer LayerNorm and softmax kernels [6], [7]; our contribution on that operator is not the kernel but the size of its disagreement with the simulator, and the demonstration that fixing it is not sufficient. Accumulator width has been studied as a training-time constraint [8]; the axis measured here is instead the rounding of the requantization step, a pure implementation choice.

III. MEASUREMENT SETUP

A. Integer engine and external validation

A calibrated fake-quantization graph is exported to an integer graph: int8 weights, int32 biases, and a per-output-channel Q31 multiplier and shift. Execution follows the `gemmlowp/TensorFlow Lite` semantics of `MultiplyByQuantizedMultiplier` [3], including the saturating rounding doubling high multiply and the rounding divide by a power of two. Two independent implementations (NumPy and C++) must agree on *every* intermediate tensor, and a third, Python-free C++ executor reproduces the same codes from a serialized file.

We validate the engine against an external oracle, not against itself. Building the integer graph from a TensorFlow Lite full-integer model’s own scales, zero-points, weights and biases, the two agree on all tensors for 1000/1000 images when convolution and pooling use the legacy double rounding and the fully-connected layer uses single rounding. For calibration

The author is an independent researcher, [CITY], Republic of Korea (e-mail: ykh13571@gmail.com).

Code, checkpoints and every result file needed to reproduce the tables and figures: <https://github.com/sokldjs554/npuloop>.

TABLE I

FAKE QUANTIZATION VERSUS BIT-EXACT INTEGER EXECUTION, COMPLETE TEST SPLITS. PC: PER-CHANNEL WEIGHTS, PT: PER-TENSOR. Δ IS (INTEGER – FAKE) TOP-1 ON THE SAME IMAGES, WITH ITS PAIRED STANDARD ERROR. “CODES” IS THE FRACTION OF OUTPUT LOGIT CODES THAT DIFFER; “IMGS” THE FRACTION OF IMAGES WITH AT LEAST ONE DIFFERING CODE.

Network	Sch.	float32	fake	integer	Δ (pp)	95% CI	agree (%)	codes (%)	imgs (%)
ResNet-20 ReLU	PC	89.59	89.54	89.55	$+0.01 \pm 0.08$	$[-0.14, +0.16]$	99.37	50.1	99
ResNet-20 ReLU	PT	89.59	89.61	89.67	$+0.06 \pm 0.08$	$[-0.11, +0.23]$	99.18	51.7	99
ResNet-20 SiLU	PC	90.34	90.35	90.26	-0.09 ± 0.09	$[-0.26, +0.08]$	99.06	61.6	100
ResNet-20 SiLU	PT	90.34	90.17	90.26	$+0.09 \pm 0.09$	$[-0.08, +0.26]$	99.07	62.9	100
MobileNetV2-0.5	PC	90.23	90.30	90.29	-0.01 ± 0.08	$[-0.16, +0.14]$	99.28	50.4	100
MobileNetV2-0.5	PT	90.23	90.23	90.27	$+0.04 \pm 0.07$	$[-0.10, +0.18]$	99.37	51.4	100
ViT-128/6	PC	80.98	80.87	80.97	$+0.10 \pm 0.13$	$[-0.15, +0.35]$	97.93	72.7	100
ViT-128/6	PT	80.98	80.77	80.97	$+0.20 \pm 0.12$	$[-0.04, +0.44]$	98.07	72.6	100
Inception-32	PC	89.59	89.55	89.56	$+0.01 \pm 0.05$	$[-0.09, +0.11]$	99.69	28.5	95
Inception-32	PT	89.59	89.52	89.47	-0.05 ± 0.06	$[-0.17, +0.07]$	99.52	31.1	96
ResNet-20 (Imagenette)	PC	83.97	84.05	83.97	-0.08 ± 0.10	$[-0.27, +0.12]$	99.52	35.7	95
ResNet-20 (Imagenette)	PT	83.97	84.00	84.05	$+0.05 \pm 0.11$	$[-0.17, +0.27]$	99.31	36.2	96

of how tight that criterion is: TensorFlow Lite’s own optimized delegate and its reference kernels differ on 155/1000 images of the same model.

B. Local and propagated mismatch

For one image, let f_n be the codes the fake-quantization graph produces at node n (its quantizer output, converted to integer codes) and i_n the codes the integer engine produces when run end-to-end. The *propagated* mismatch at n is the fraction of elements with $f_n \neq i_n$. The *local* mismatch feeds node n ’s integer kernel the fake-quantization codes of its inputs (teacher forcing) and compares only its own output. Local mismatch isolates what an operator’s integer arithmetic contributes; propagated mismatch shows how far that contribution travels.

C. Statistics, models and schemes

Accuracy differences are paired: the same images are evaluated three times (float, fake-quantization, integer). We report the mean per-image difference in correctness with its standard error $s/\sqrt{n}$ and a 95% Wald interval. Accuracy columns use the complete test split (10,000 images for CIFAR-10, 3,925 for Imagenette); the per-node decomposition uses a fixed batch of 250 images (125 for Imagenette).

Five CIFAR-10 networks — ResNet-20 with ReLU and with SiLU, MobileNetV2-0.5 (depthwise), a concatenation-branch Inception-style CNN, and a ViT-128/6 (12 attention blocks, 13 LayerNorms) — and one ResNet-20 trained on Imagenette at 128×128 . Two post-training schemes: per-channel weights with asymmetric activations (PC) and per-tensor weights (PT), both calibrated on 512 training images. Training and evaluation run on CPU; every number below is emitted by a script from a committed results file.

IV. RESULTS

A. RQ1: accuracy agrees, tensors do not

Table I is the main measurement. The largest fake-versus-integer accuracy difference is 0.20 pp, the largest $|\Delta|/\text{SE}$ is 1.6, and all twelve confidence intervals contain zero. At these sample sizes the accuracy proxy is not distinguishable from

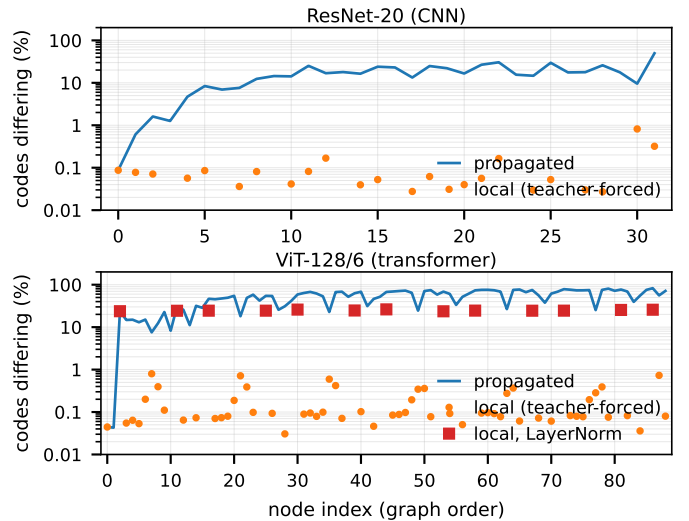


Fig. 1. Per-node code mismatch, PC scheme. Local (teacher-forced) mismatch is below 1% for every operator except LayerNorm; propagated mismatch rises from the first convolution to 50–73% at the output. Log scale; nodes with zero local mismatch (lookup-table activations, residual additions) are not drawn.

exact, and image-level top-1 agreement is 97.9–99.7%. The same rows disagree on tensors: between 28.5% (Inception-32) and 72.7% (ViT-128/6) of the output logit codes differ, on 95–100% of the images. The transformer is the extreme case but not a special case — the CNNs sit at 50–63% and the larger-input Imagenette model at 36%.

Finding 1. Accuracy agreement and tensor agreement are decoupled by more than two orders of magnitude, on every network and both schemes.

Implication. A reported 99% accuracy agreement is no evidence that the simulator can supply a verification vector.

B. RQ2: where the divergence is born

Fig. 1 separates the two components. Local mismatch is small and flat: convolution disagrees on 0.02–0.93% of its codes, linear on 0.03–0.96%, average pooling on 0.1–2.7% (rounding ties), and the int8 lookup-table activations on exactly 0%. Every one of these is a ± 1 least-significant-bit disagreement. Propagated mismatch instead rises monotonically from

TABLE II
MAKING THE SIMULATOR INTEGER-FAITHFUL AT LAYER NORM
(ViT-128/6, 10,000 IMAGES). THE INTEGER PROGRAM IS UNCHANGED
BETWEEN THE TWO ROWS OF EACH SCHEME; ONLY THE SIMULATOR'S
LAYER NORM DIFFERS. THE MEAN ABSOLUTE LOGIT DIFFERENCE FALLS
FROM 0.119 TO 0.091 (PC) AND FROM 0.118 TO 0.093 (PT).

Sch.	LayerNorm	local (%)	codes (%)	agree (%)	Δ (pp)
PC	float32	24.77	72.7	97.93	$+0.10 \pm 0.13$
PC	integer (13)	0.00	65.7	98.29	$+0.25 \pm 0.11$
PT	float32	24.76	72.6	98.07	$+0.20 \pm 0.12$
PT	integer (13)	0.00	66.0	98.60	$+0.13 \pm 0.10$

the first convolution — the first divergence is that node in all twelve rows — and reaches 28–73% at the output.

One operator is not small. Integer LayerNorm disagrees with its floating-point counterpart on 24.8% of its codes, with a maximum of 26.2% over the 13 instances. The integer implementation normalizes with an int64 sum and an exact integer square root, while the simulator computes in float32 and then snaps to the grid, so roughly one element in four lands on the other side of a rounding boundary.

Finding 2. Outside LayerNorm, every operator's own integer arithmetic disagrees on under 1% of its codes, and every such disagreement is ± 1 LSB. The end-to-end gap is therefore grown by propagation, not contributed by any one kernel.

Finding 3. Integer LayerNorm is the single large local source, two orders of magnitude above every other operator.

Implication. When porting a transformer to INT8, suspect LayerNorm first and expect ± 1 LSB everywhere else.

C. RQ2: removing the largest local source

If the divergence were a sum of per-operator errors, removing the largest would help proportionally. We test this directly: the simulator's LayerNorm is replaced by a module that computes the engine's arithmetic on the input codes, bit-exact by construction, while the exported integer program is unchanged (the experiment asserts this).

Table II shows the outcome. Local LayerNorm mismatch goes from 24.77% to exactly 0.00%, and the mean absolute logit difference improves by about 23%. The propagated output-code mismatch moves only from 72.7% to 65.7% (per-tensor: 72.6% $\rightarrow$ 66.0%). Accuracy moves within about two paired standard errors in either direction and cannot be read as an improvement.

Finding 4. Eliminating the single largest local source recovers about a tenth of the propagated gap, because what remains is the amplification of ± 1 LSB events from operators that individually disagree on under 1% of their codes.

Implication. Golden vectors have to come from an integer implementation validated against an external reference. Repairing a floating-point simulator operator by operator converges too slowly to substitute for one.

D. RQ3: a cost the accuracy proxy hides

The requantization multiply-and-shift ends in a rounding step, and an implementation may be tempted to replace it with

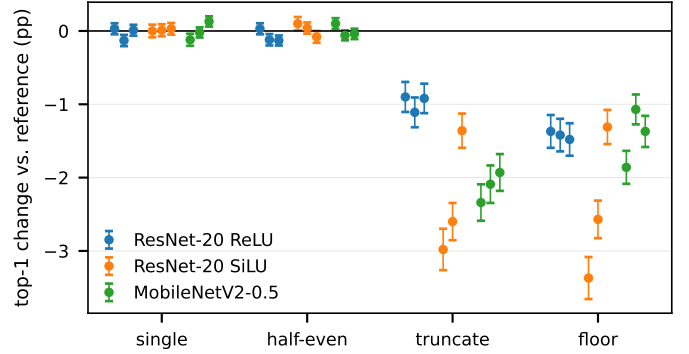


Fig. 2. Top-1 change against the reference requantization rounding, complete test split, three independently trained checkpoints per network (error bars: paired standard error). The biased modes lose in every combination; how much they lose varies by seed.

TABLE III
ROUNDING-MODE COST AGAINST THE REFERENCE IMPLEMENTATION,
MEAN $\pm$ STANDARD DEVIATION OVER THREE SEEDS (PP).

Mode	ResNet-20 ReLU	ResNet-20 SiLU	MobileNetV2-0.5
single	-0.03 ± 0.09	$+0.01 \pm 0.02$	-0.00 ± 0.13
half_even	-0.07 ± 0.09	$+0.02 \pm 0.09$	$+0.00 \pm 0.09$
truncate	-0.98 ± 0.12	-2.31 ± 0.85	-2.12 ± 0.21
floor	-1.42 ± 0.06	-2.42 ± 1.04	-1.43 ± 0.40

a plain arithmetic shift. Fig. 2 and Table III price that choice against the reference implementation on full test splits, with three independently trained checkpoints per network.

`truncate` and `floor` are worse than the reference in all 18 (model, seed, mode) combinations, by 0.90–3.37 pp, with $|\Delta|/\text{SE}$ between 4.4 and 11.8; the alternative single-rounding and round-half-to-even paths stay within noise ($|\Delta| \leq 0.13$ pp). The magnitude does not transfer. For ResNet-20 SiLU the cost of `truncate` ranges from 1.36 to 2.98 pp across three seeds, a seed standard deviation equal to 37% of the effect, while ResNet-20 ReLU stays within a tenth of its effect. Which biased mode is worse is a property of the network, not of the arithmetic: `floor` loses more on all three ResNet-20 ReLU seeds, `truncate` on all three MobileNetV2 seeds.

Finding 5. The sign and significance of a rounding-mode cost are seed-robust; its magnitude is not, varying by up to 37% of the effect across three seeds of the same architecture.

Implication. Do not quote a single-seed number as the cost of a rounding mode.

V. THREATS TO VALIDITY

External. The networks are CIFAR-10 scale plus one Imagenette model at 128×128 , and all six are classifiers. The conclusions are about the structure of the divergence rather than its exact size on ImageNet-scale graphs, and a dense-output task — where the tensor, not an argmax over it, is the deliverable — would expose the tensor gap more directly than accuracy does here.

Construct. Local mismatch is teacher-forced, so it measures an operator's arithmetic in isolation and not its interaction with

upstream error; the propagated column is what captures that interaction.

Conclusion. The twelve accuracy comparisons in Table I and the four in Table II are not corrected for multiple comparisons; over all sixteen the largest $|\Delta|/\text{SE}$ is 2.2. Three seeds cover the rounding axis only, and the remaining measurements use one checkpoint per network.

Internal. The external bit-exactness check covers convolution, pooling and fully-connected layers; depthwise convolution, element-wise addition and softmax are not covered by it.

VI. ARTIFACT AVAILABILITY

Every number is emitted by a script from a committed result file, and the scripts, the trained checkpoints and those result files are public. `make_tables.py` regenerates Tables I–III and `make_figs.py` regenerates Figs. 1–2; a continuous-integration job re-runs the generators on every commit and fails if a committed table differs from the one the results imply. The measurements are `e15_fidelity.py` (Table I, Fig. 1), `e16_ln_emulation.py` (Table II) and `e14_rounding_seeds.py` (Fig. 2, Table III); the external check is `e11_tflite_crosscheck.py`. One caveat applies to the seeded measurement: the trainer’s seed controlled data order but not weight initialization when these checkpoints were produced, so the three replicates are independent draws that the same command will not reproduce bit for bit.

VII. CONCLUSION

Simulated quantization predicts INT8 accuracy well and INT8 tensors badly, and the two facts are independent. The gap is not carried by any single operator: it is born as ± 1 LSB events in operators that individually disagree on under 1% of their codes, and grown by propagation, which is why removing the worst local source recovers only a tenth of it. Teams that need golden vectors should generate them from an integer implementation validated against an external reference, and teams that change a requantization detail should measure it across seeds, because its magnitude — unlike its sign — does not transfer.

ACKNOWLEDGMENT

The author used an AI coding assistant (Claude Code) while preparing this manuscript and is responsible for all of its content.

REFERENCES

- [1] B. Jacob *et al.*, “Quantization and training of neural networks for efficient integer-arithmetic-only inference,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2018, pp. 2704–2713.
- [2] R. Krishnamoorthi, “Quantizing deep convolutional networks for efficient inference: A whitepaper,” *arXiv:1806.08342*, 2018.
- [3] B. Jacob *et al.*, “gemmlowp: a small self-contained low-precision GEMM library,” 2017. [Online]. Available: <https://github.com/google/gemmlowp>
- [4] M. Nagel *et al.*, “A white paper on neural network quantization,” *arXiv:2106.08295*, 2021.
- [5] Y. Li *et al.*, “MQBench: Towards reproducible and deployable model quantization benchmark,” in *Proc. NeurIPS Datasets and Benchmarks Track*, 2021.
- [6] S. Kim *et al.*, “I-BERT: Integer-only BERT quantization,” in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2021, pp. 5506–5518.
- [7] Y. Lin *et al.*, “FQ-ViT: Post-training quantization for fully quantized vision transformer,” in *Proc. Int. Joint Conf. Artif. Intell. (IJCAI)*, 2022, pp. 1173–1179.
- [8] I. Colbert *et al.*, “A2Q: Accumulator-aware quantization with guaranteed overflow avoidance,” in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2023, pp. 16989–16998.